Assignment 1 - Report For Part 1 & 2

Taha Razzaq - 24030023

Github repository where the code resides: https://github.com/TahaRazzaq/DL4STG

Part 1:

Response 1A. Predict, measure, and revise the aggregate ordering.

`Before running prediction:`

Since shared ridge and sensor-specific ridge use fixed forecasting rules and do not adapt their forecasting technique to accomodate current model pace, hence I expect their performance to be the worst, among these two sensor-specific ridge will perfrom slightly better since shared ridge uses a single weight matrix for all the forecasting which is not a good technique. Period-routed ridge would perform much better than these 2 techniques because it uses peridoic information which means that the model is specially conditions to quantify the periodic changes. Lastly, the raw attention will perfrom best because it learns the weights on the go and hence adapts to learning temporal relationships which are essential for forecasting

Predicted Ordering (worst to best):

- Shared Ridge
- Sensor Specific Ridge
- Period Routed Ridge
- Raw Attention.

With 4th station (Raw attention) held out, the remaining ordering is predicted to remain the same

`Measurements:`

Looking at the output 1.1, we observed that the predicted ordering and the actual ordering are poles apart. We observed that established stations RMSE (m/s²) for period routed ridge was actually the smallest and was closely follwoed by shared ridge and sensor specific ridge. As opposed to our predictions, raw attention performed the worst. Period Routed Ridge achieved an RMSE score fof 0.166 for established station and even for held-out station 4, Product A, B and C it achived the lowest RMSE score (0.173). Raw attention with 0.421 RMSE followed as 2nd best perfroming models for all cases while Shared ridge with 0.767 RMSE had the worst performance out of all 4 models.

`Revised ordering:`

Our initial ordering for worst to best model performance was

Shared Ridge → Sensor Specific Ridge → Period Routed Ridge → Raw Attention

however looking at the results we revise our ordering and state the updated one as

Shared Ridge → Sensor Specific Ridge → Raw Attention → Period Routed Ridge

for worst to best performing model. This shows that the ability to recompute attention weights from context is useful but not as compared to implicitly encoding periodic information. In this case periodic information allows for the model to better learn the temporal trend as compared to learnable attention weights.

Response 1B. Explain why the forecasting rule must depend on context.

`Ouptut 1.2 analysis:`

Using the output 1.2 we observed that in Product A we had 16.135 samples as compared to 30.093 samples in product C and the Period-routed ridge model performed much better in both Product C and product A however the perfromance in Product C was significantly better. Looking at the output graph of output 1.2 we also observe that the peak distribution of product A and Product C are extremely different. Product C peaks are distributed while product A has peaks close to each other which means the model is supposed to learn this distribution independantly and hence using the same model for 2 different products with 2 different peak and phase distributions show that a single weight matrix is not suitable to be used. The compromise is that a single linear combination learnt on a specific forecasting phase cannot be applied to another with a different phase. We observed this trend for all the models that the performance in product C was much better than product A.

`Ouptut 1.3 analysis:`

Using the output 1.3 we observed that for product A vs C mean |Attention weight difference| = 0.011 and max |Attention weight difference| = 0.517 which clearly indicates that the attention was able to produce context dependant operators. Although the projects of Query, key and vlaue matrices were learnt during training, when applied to different products they used the input as context and were able to produce context aware Query, key and value vectors, hence varying attention weights, which allowed for the difference between two products to be low.

This is different from ridge since in ridge the same weight vector was applied to differnt inputs, however in attention context aware weights are generated and applied to the input.

Part 2:

`State the selected width and describe what changes visually in Output 2.1 and empirically in Outputs 2.2–2.3, including the validation comparison of Raw Attention and Attention plus decomposition`

I implemented the decomposition using moving average with replicate padding where the number was replicated as a form of padding instead of padding with zero, then we computed the trend and remiander using remainder = x - trend formula. The output 2.1 show that as the width of the moving average window increases from 17 to 41, the variation in the remainder also increases. The residual std of width 41 is the highest with 1.572 and hence that is the width that gets selected. This basically is intuitiev since larger windows encapsulate a smoother trend and hence the variation in the remiander is increased.

In output 2.2, we observe the effect of Attention with decomposition. We notice that for all the widths with decomposition the results imporve and for width 41 (the selected one) recoves a

perfect score with only a single sample, while other widths also follow close behind. This clearly shows that decomposition is successfully removing the slow drifts which makes the periodic structure easy to identify and hence forecast.

Looking at output 2.3, we observed that attention + decomposition had a higher MSE and RMSE score as compare to Raw attention for all products and combination. This shows that while decomposition is able to successfully identify the temporal patterns it isn't improving the forecasting results and as compared raw attention is much better.

`Explain why a moving average is a sensible inductive bias for this drift-plus-repetition world, and why the training-only oracle diagnostic is a useful way to select its width here`

Since the data has a slow drift and fast vibrations, using moving average is a sensible inductive bias since it is good at seperating parts from the data with varying changes. It basically smooths the fast vibrations and preserves the slow changes hence it is a good option.

For using the training-only oracle diagnostic for selecting the width is useful because we know how the training data will look like and we are using it to evaluate the widths and select one accordingly.

`Give one situation where this bias would fail or name one alternative decomposition and the different assumption it would encode.`

The moving average inductive bias is based on a crucial assumption that the data has slow and then fast moving trends which can easily be distinguished but if the vibrations become slower i.e their intensities decrease, there is a possiblity the moving average might treat them as the slow drift part and hence the temporal data structure learnt will be flawed.

Instead of using a simple moving average we can use a seasonal decomposition like STL which takes into account the seasonality within the trend. It uses local weighted averaging on a trend and this allows for the trend per season to be learnt.

Part 3:

- `Which lags show the strongest recurrence in the diagnostic, and how do they relate to that operating run’s actual period P and its integer multiples?`

The operating run's actual period P is 16 and we observe that for 16 and 32 (which is 2P) a correlation of 0.982 is observed which is a strong correlation and hence idicates that the vibration cycle, even after shifting 1 or 2 cycles from its original cycle is still very similar. The strongest recurrence occur at a delay of 16 and 32 which is P and 2P respectively hence indicative of the fact that strong recurrence ocurr at P or its integers multiples.

- `Why does decomposing the context first (removing the slow drift) make the delay evidence cleaner than in the raw signal, and why does that support the delay mixer’s inductive bias?`

The context had slow drift and fast vibrations, and when a cycle was offsetted the slow drift would act as noise making it harder for the model to learn the fast vibration trends, hence removing the slow drift via decomposition allows for the fast vibration trends to be captured

quickly and accurately. This directly supports the delay mixer's inductive bias which states that instead of looking at all the time points, we simply look at the important ones which are P and its integer multiples. If we hadn't removed the slow drift then under the delay mixers inductive bias, we might have looked at a potentially import time point but would have captured the slow changes instead of the fast vibrations.

- `Delay mixing assumes that useful temporal relationships are organized by a small set of recurring displacements. Describe one situation where this assumption would be the wrong one to build in — for example, an aperiodic event, a period that drifts quickly within one context, or an isolated transient — and explain briefly why delay-based mixing would mishandle it.`

Delay mixers assumes that useful temporal events will always occurr at a fixed displacements, for example in this run at an interval of P, however if an important event was to occur at a timestamp before the multiple of P, then that would be missed and hence not noted in the temporal trend. Another example is that if a period drifts within one context i.e the period changes form 16, 20 and 25 samples then using a fixed displacement would mishandle it.

Part 4:

`Prediction:`

The decomposition + delay mixer (Autoformer inspired) model would work the best since we know that slow parts are prominent in the data and hence removing the slow drift prior to learning the temporal sequence would be the best. Morever, delay mixer can aid in learning the temporal sequence prior to decomposition

`From Outputs 4.1 and 4.2, report the validation RMSEs and MSEs for the four neural models and the linear baselines on the established stations; compare the two architectural switches in both settings, and summarize how errors change across the forecast horizon`

Table 1: Established station | Model | RMSE | MSE | | ---------- | --------: | --------: | | Raw Attention | 0.421 | 0.178 | | Raw delay mixer | 0.213 | 0.046 | | Attention + decomposition | 0.293 | 0.086 | | Autoformer-inspired | 0.199 | 0.040 | | Shared ridge | 0.767 | 0.5887 | | Sensor-specific ridge | 0.768 | 0.590 | | Period-routed ridge |**0.166** | **0.028** |

Table 2: Held-out station 4 | Model | Held-out RMSE | Held-out MSE | | ---------- | ----------: | ----------: | | Raw Attention | 0.608 | 0.369 | | Raw delay mixer | 0.339 | 0.115 | | Attention + decomposition | 0.436 | 0.190 | | Autoformer-inspired | 0.240 | 0.058 | | Shared ridge | 0.876 | 0.767 | | Sensor-specific ridge | unavailable | unavailable | | Period-routed ridge | **0.173** | **0.030** |

Among the deeplearning baselines, Autoformer inspired model has the lowest RMSE and MSE and hence the best performance on the established station aggregate and for the held-out station 4. Looking at the results we observed that raw attention has the worst performance among all deelearning and classical baselines, however raw delay mixer in comparison performed substantially better. We further noticed that even combining raw attention and decomposition does not give the desired effect and it still underperforms raw delay mixer, hence combining raw delay mixer with decomposition would boost the results. We observed exactly

this in the sutoformer inspired model which outperformed all the other DL baselines. Moreover, the autoformer inspired model has better performance than raw delay mixer alone, this is indicative that these architectural changes might not give us significant gain independantly as they do when combined together.

However when we compare these models with classical baselines, established in part 1, we observed that the baselines actually have lower rmse and mse score, infact for both established and heldout sations **period routed ridge** outperforms all the other models including the autoformer and raw attention.

For the horizon-wise results we observed that it gets difficult to forecast the data as the horizon increases even though the error does not rise monotonically. For example, the period routed ridge rmse increases from 0.112 at horizon 1 to 0.121 at horizon 5. At longer horizons, larger fluctuations can occur. For the Autoformer-inspired model on the held-out station, the rmse values at horizons 44 till 48 are 0.315, 0.326, 0.313, 0.320 and 0.301 respectively. Thus, increasing horizon generally reduces predictability, but the periodic structure can produce non-monotonic error patterns.

`Based on validation performance, parameter count, and fitting time, identify the best deployment model for the established stations and for held-out Station 4, and justify each choice.`

Based on the results shown in the above anlayis + notebook outputs, we observed that Period-routed ridge has the lowest rmse and mse for both established and held-out station and hence that is the natural choice for both. If we look at the neural networks we tested in Part 4, the results show that autoformer-inspired model would be the best choice out of the 2x2 choices for the established stations and for held-out station 4, raw attention would be selected. These selections are primarily based on their RMSE and MSE scores.

I would go ahead with Period-routed ridge for both established stations and held-out station 4 because firstly this baseline has a significantly lower rmse and mse score as compared to both Autoformer inspired and raw attnetion model. Secondly, if we compare the fitting time, Period-routed ridge took the least time with only 0.015 seconda while autoformer inspired and raw attention took 1089.873 and 42317.004 seconds respectively. Lastly, the number of parameters in Period-routed ridge model are substantially lower than the other two models. Period-routed ridge has 59904 parameters as opposed to 368368 parameters of raw attention and 373026 parameters for autoformer-inspired model. Considering the performance, parameter count and fitting time, Period-routed ridge model is the best deployment model choice for both established stations and held-out station 4.

Task 2:

For this part I used a pre-existing Autoformer implementation (adapted from https://github.com/thuml/Autoformer.git) along with the training, testing and optional external data. I split the training data into train and validation sets by keeping the last 1008 values (6 × 168) for validation. On that block I scored rolling 168-step forecasts that start 24 steps apart, because a single window was too noisy to compare runs. Initially, only the training data was used to assess the model performance for the time forecasting. However, when I ran the ablation study to determine if the external data would add value, I observed that the additional covariates were boosting the performance, since their future values are known over the forecast

horizon. Hence for the next set of experiments, I tested the number of external features to add along with the training data to boost the model's performance on the testing set. I started with 4 features and then scaled up to 40 features, which gave the best results. Along with the 10 raw features, I added a log of the heavy-tailed feature D, the B − A spread, hour-of-day sine/cosine terms for the daily cycle, rolling means over 6, 24 and 72 hours, and 24-hour differences. All features were standardized using training-set statistics.

For the Autoformer, I set the input context to 336 and the output to 168 given our problem statement, with a label length of 168 so the decoder starts from the last week of known values. The moving average window was kept at 25, which is the odd kernel size closest to the 24-hour cycle, so the trend smooths out one day of seasonality. I kept the model small, with d_model = 64, 4 attention heads, 2 encoder layers, 1 decoder layer, d_ff = 128, GELU activation, dropout of 0.1 and an auto-correlation factor of 1. Since the training data is a single series, a larger model mostly overfit, and most of the signal came from the covariates anyway. Instead of the default timeF calendar features, I replaced the temporal embedding with a linear layer and passed the external covariates through it. This way the decoder sees the known future covariate values without any change to the core architecture. For training, I used MSE loss since the leaderboard metric is RMSE, and the Adam optimizer with a learning rate of 1e-3, weight decay of 1e-5 and a batch size of 64.

The target was standardized using the training-set mean and standard deviation. I also tried per-window instance normalization, but it removes the absolute level, which stops the covariates from setting the forecast level, so global scaling did better on validation. Forecasts were clipped at 0 since the target is non-negative. The initial model was trained for 8 epochs on the training data only. It was the lightest, but it yielded the worst score (95.5520) on the leaderboard. For the second run, the same model was rerun with the added covariates, and that improved the results. For the next runs, I created an ensemble by averaging forecasts from 5 seeds trained for 12 epochs each, retrained on the full history (train + validation). This improved the results significantly and the score dropped to 69.8130. The parameters and number of epochs increased, but their penalty on the score was negligible compared to the RMSE gain.